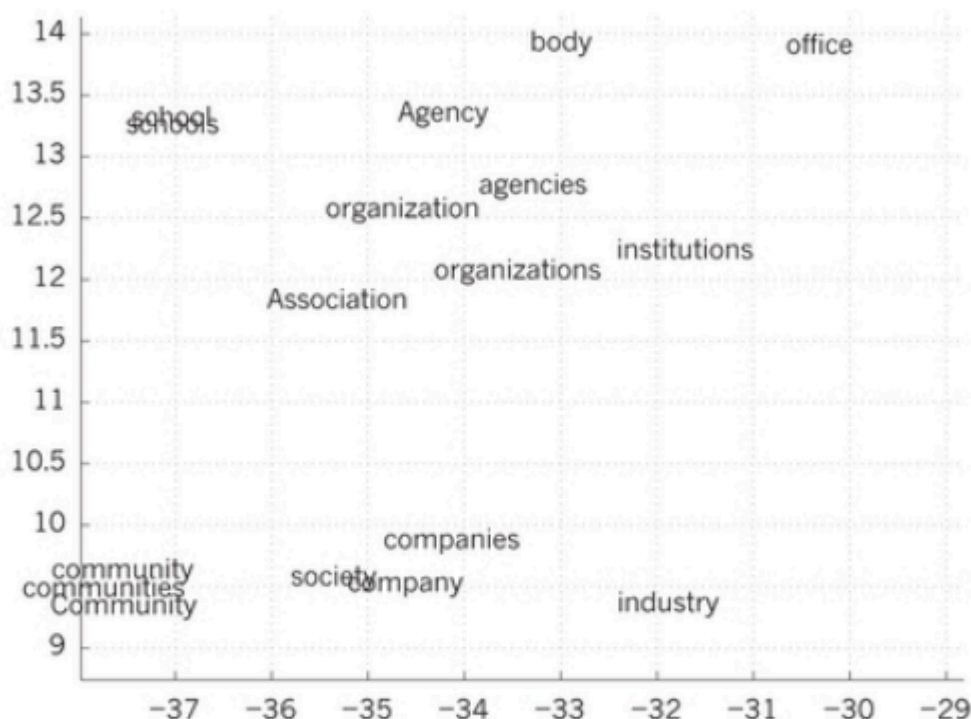


## Векторные представления слов (word embeddings)

Векторное представление слова (word embedding) — вещественный вектор в пространстве с фиксированной невысокой размерностью.

Пример векторных представлений слов (2D

t-SNE)



Deep learning, Yann LeCun, Yoshua Bengio & Geoffrey Hinton, Nature 521, 436–444 (28 May 2015) | doi:10.1038/nature14539

Зачем нужны Word embeddings? Сжатые векторные представления слов

1. полезны сами по себе, например, для поиска синонимов или опечаток в поисковых запросах.
2. используются в качестве признаков для решения самых различных задач: выявление именованных сущностей, тэги́рование частей речи, машинный перевод, кластеризация документов, ранжирование документов, анализ тональности текста.

### word2vec

Мера семантической близости — мера близости, предназначенная для количественной оценки семантической схожести слов. Такая мера показывает высокие значения для пар слов, находящихся в семантических отношениях (синонимия, ассоциативность и т.д.), и нулевые значения для всех остальных пар.

word2vec - алгоритм для получения векторных представлений слов. Подход основан на важной гипотезе, которую в науке принято называть гипотезой локальности — “слова, которые встречаются в одинаковых окружениях, имеют близкие значения”. Близость в данном случае понимается очень широко, как то, что рядом могут стоять только сочетающиеся слова. Например, для нас привычно словосочетание "заводной будильник". А сказать "заводной апельсин" мы не можем\* — эти слова не сочетаются.

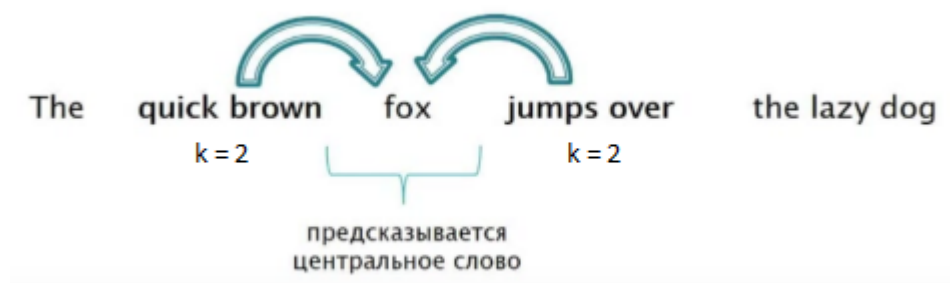
## Алгоритм word2vec

Мы будем предсказывать вероятность слова по его окружению (контексту). То есть мы будем учить такие вектора слов, чтобы вероятность, присваиваемая моделью слову была близка к вероятности встретить это слово в этом окружении в реальном тексте.

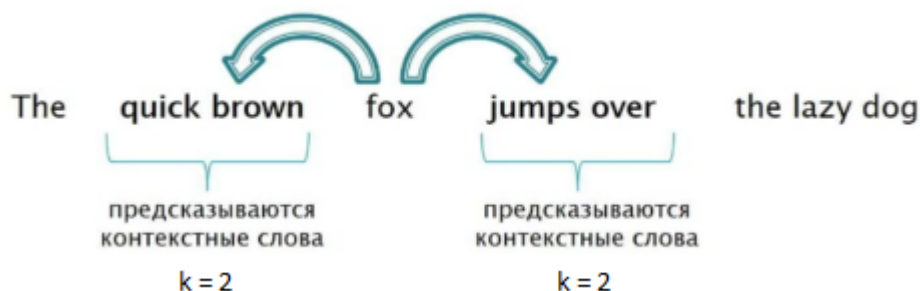
$$P(w_o|w_c) = \frac{e^{s(w_o, w_c)}}{\sum_{w_i \in V} e^{s(w_i, w_c)}}$$

Здесь  $w_o$  — вектор целевого слова,  $w_c$  — это некоторый вектор контекста, вычисленный (например, путем усреднения) из векторов окружающих нужное слово других слов. А  $S$  — это функция, которая двум векторам сопоставляет одно число. Например, это может быть косинусное расстояние.

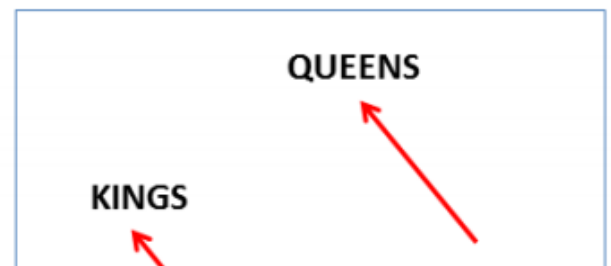
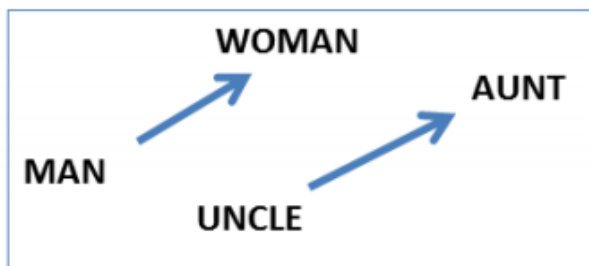
Процесс тренировки устроен следующим образом: мы берем последовательно  $(2k+1)$  слов, слово в центре является тем словом, которое должно быть предсказано. А окружающие слова являются контекстом длины по  $k$  с каждой стороны. Каждому слову в нашей модели сопоставлен уникальный вектор, который мы меняем в процессе обучения нашей модели. В целом, этот подход называется CBOW — continuous bag of words, continuous потому, что мы скормливаем нашей модели последовательно наборы слов из текста, а BoW потому что порядок слов в контексте не важен.



Другой подход skip-gram — прямо противоположный CBOW, то есть “словосочетание с пропуском”. Мы пытаемся из данного нам слова угадать его контекст (точнее вектор контекста). В остальном модель не претерпевает изменений.



Что стоит отметить: хотя в модель не заложено явно никакой семантики, а только статистические свойства корпусов текстов, оказывается, что натренированная модель word2vec может улавливать некоторые семантические свойства слов. Классический пример:



### Что мы можем попробовать сделать с векторами слов?

Мы можем делать различные синтаксические, семантические NLP задачи с векторами слов, некоторое из них уже встроены.

```
Ввод [1]: import gensim.downloader as api
```

```
C:\ProgramData\Anaconda3\lib\site-packages\scipy\__init__.py:155: UserWarning: A NumPy version >=1.18.5 and <1.25.0 is required for this version of SciPy (detected version 1.26.2)
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
```

```
Ввод [2]: api.info()['models'].keys()
```

```
Out[2]: dict_keys(['fasttext-wiki-news-subwords-300', 'conceptnet-numberbatch-17-06-300', 'word2vec-ruscorpora-300', 'word2vec-google-news-300', 'glove-wiki-gigaword-50', 'glove-wiki-gigaword-100', 'glove-wiki-gigaword-200', 'glove-wiki-gigaword-300', 'glove-twitter-25', 'glove-twitter-50', 'glove-twitter-100', 'glove-twitter-200', '__testing_word2vec-matrix-synopsis'])
```

```
Ввод [ ]:
```

```
Ввод [3]: import gensim.downloader as api
```

```
word_vectors = api.load("glove-wiki-gigaword-100") # загрузим предтренированные векторы
# выводим слово наиболее близкое к 'woman', 'king' и далекое от 'man'
result = word_vectors.most_similar(positive=['woman', 'king'], negative=['man'])
print("{}: {:.4f}".format(*result[0]))
```

```
[=====] 100.0% 128.1/128.1MB downloaded
queen: 0.7699
```

```
Ввод [4]: # выведем лишнее слово
print(word_vectors.doesnt_match("breakfast cereal dinner lunch".split()))

print(word_vectors.doesnt_match("black green summer brown".split()))
```

```
cereal
summer
```

```
Ввод [5]: # определим схожесть между словами
similarity = word_vectors.similarity('woman', 'man')
print(similarity)

similarity = word_vectors.similarity('human', 'man')
print(similarity)

similarity = word_vectors.similarity('bee', 'man')
print(similarity)
```

```
0.8323494
0.5288512
0.21199903
```

```
Ввод [6]: # найдем top-3 самых близких слов
result = word_vectors.similar_by_word("man", topn=3)
print(result)

result = word_vectors.similar_by_word("cat", topn=3)
print(result)

result = word_vectors.similar_by_word("mouth", topn=3)
print(result)
```

```
[('woman', 0.8323495388031006), ('boy', 0.7914870977401733), ('one', 0.7788748741149902)]
[('dog', 0.8798074722290039), ('rabbit', 0.7424427270889282), ('cats', 0.732300341129303)]
[('tongue', 0.7366125583648682), ('mouths', 0.6877489686012268), ('ear', 0.6811771392822266)]
```

## Simple chat-bot example

```
Ввод [15]: #!pip install pymorphy2  
#!pip install stop_words  
!pip install annoy  
!pip install pickle
```

Defaulting to user installation because normal site-packages is not writeable

Collecting annoy

Using cached annoy-1.17.3.tar.gz (647 kB)

Preparing metadata (setup.py): started

Preparing metadata (setup.py): finished with status 'done'

Building wheels for collected packages: annoy

Building wheel for annoy (setup.py): started

Building wheel for annoy (setup.py): finished with status 'error'

Running setup.py clean for annoy

Failed to build annoy

Installing collected packages: annoy

Running setup.py install for annoy: started

Running setup.py install for annoy: finished with status 'error'

error: subprocess-exited-with-error

python setup.py bdist\_wheel did not run successfully.  
exit code: 1

```
[12 lines of output]
running bdist_wheel
running build
running build_py
creating build
creating build\lib.win-amd64-cpython-39
creating build\lib.win-amd64-cpython-39\annoy
copying annoy\__init__.py -> build\lib.win-amd64-cpython-39\annoy
copying annoy\__init__.pyi -> build\lib.win-amd64-cpython-39\annoy
copying annoy\py.typed -> build\lib.win-amd64-cpython-39\annoy
running build_ext
building 'annoy.annoylib' extension
error: Microsoft Visual C++ 14.0 or greater is required. Get it with "Microsoft C++
Build Tools": https://visualstudio.microsoft.com/visual-cpp-build-tools/ (https://visualstudio.microsoft.com/visual-cpp-build-tools/)
[end of output]
```

note: This error originates from a subprocess, and is likely not a problem with pip.  
ERROR: Failed building wheel for annoy  
error: subprocess-exited-with-error

Running setup.py install for annoy did not run successfully.  
exit code: 1

```
[14 lines of output]
running install
C:\ProgramData\Anaconda3\lib\site-packages\setuptools\command\install.py:34: SetuptoolsDeprecationWarning: setup.py install is deprecated. Use build and pip and other standards-based tools.
  warnings.warn(
running build
running build_py
creating build
creating build\lib.win-amd64-cpython-39
creating build\lib.win-amd64-cpython-39\annoy
copying annoy\__init__.py -> build\lib.win-amd64-cpython-39\annoy
copying annoy\__init__.pyi -> build\lib.win-amd64-cpython-39\annoy
copying annoy\py.typed -> build\lib.win-amd64-cpython-39\annoy
running build_ext
building 'annoy.annoylib' extension
error: Microsoft Visual C++ 14.0 or greater is required. Get it with "Microsoft C++
Build Tools": https://visualstudio.microsoft.com/visual-cpp-build-tools/ (https://visualstudio.microsoft.com/visual-cpp-build-tools/)
[end of output]
```

note: This error originates from a subprocess, and is likely not a problem with pip.  
error: legacy-install-failure

Encountered error while trying to install package.

annoy

note: This is an issue with the package mentioned above, not pip.  
hint: See above for output from the failure.

Defaulting to user installation because normal site-packages is not writeable

ERROR: Could not find a version that satisfies the requirement pickle (from versions: none)  
ERROR: No matching distribution found for pickle

```
Ввод [16]: import string
from pymorphy2 import MorphAnalyzer
from stop_words import get_stop_words
import annoy
from gensim.models import Word2Vec, FastText
import pickle
import numpy as np
from tqdm import tqdm_notebook
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_17652\2442047960.py in <module>
      2 from pymorphy2 import MorphAnalyzer
      3 from stop_words import get_stop_words
----> 4 import annoy
      5 from gensim.models import Word2Vec, FastText
      6 import pickle
```

**ModuleNotFoundError**: No module named 'annoy'

```
Ввод [ ]: !ls
```

```
Ввод [ ]: !head -5 prepared_answers.txt
```

```
Ввод [ ]: counter_all = 0
counter_filter = 0
with open("prepared_answers.txt", "r") as f:
    for line in tqdm_notebook(f):
        counter_all +=1
        spls = line.split("\t")
        if len(spls[0].split()) < 2 or len(spls[1].split()) < 3 or len(spls[0].split())
            continue

        counter_filter +=1
```

```
Ввод [ ]: assert True

#Small preprocess of the answers

question = None
written = False

with open("prepared_answers.txt", "w") as fout:
    with open("Otvety.txt", "r") as fin:
        for line in tqdm_notebook(fin):
            if line.startswith("---"):
                written = False
                continue
            if not written and question is not None:
                fout.write(question.replace("\t", " ").strip() + "\t" + line.replace("\t", " "))
                written = True
                question = None
                continue
            if not written:
                question = line.strip()
                continue
```

```
Ввод [ ]: def preprocess_txt(line):
    spls = "".join(i for i in line.strip() if i not in exclude).split()
    spls = [morpher.parse(i.lower())[0].normal_form for i in spls]
    spls = [i for i in spls if i not in sw and i != ""]
    return spls
```

```
Ввод [ ]: assert True
```

```
# Preprocess for models fitting

sentences = []

morpher = MorphAnalyzer()
sw = set(get_stop_words("ru"))
exclude = set(string.punctuation)
c = 0

with open("Otvety.txt", "r") as fin:
    for line in tqdm_notebook(fin):
        spls = preprocess_txt(line)
        sentences.append(spls)
        c += 1
        if c > 100000:
            break
```

```
Ввод [ ]: sentences = [i for i in sentences if len(i) > 2]
```

```
Ввод [ ]: sentences[0]
```

```
Ввод [ ]: Word2Vec?
```

```
Ввод [ ]: modelW2V = Word2Vec(sentences=sentences, vector_size=300, window=5, min_count=1)
```

```
Ввод [ ]: modelFT = FastText(sentences=sentences, vector_size=300, min_count=1, window=5, workers=
```



```

Ввод [ ]: w2v_index = annoy.AnnoyIndex(300, 'angular')
          ft_index = annoy.AnnoyIndex(300, 'angular')

          index_map = {}
          counter = 0

          with open("prepared_answers.txt", "r") as f:
              for line in tqdm_notebook(f):
                  n_w2v = 0
                  n_ft = 0
                  spls = line.split("\t")
                  index_map[counter] = spls[1]
                  question = preprocess_txt(spls[0])

                  vector_w2v = np.zeros(300)
                  vector_ft = np.zeros(300)
                  for word in question:
                      if word in modelW2V.wv:
                          vector_w2v += modelW2V.wv[word]
                          n_w2v += 1
                      if word in modelFT.wv:
                          vector_ft += modelFT.wv[word]
                          n_ft += 1
                  if n_w2v > 0:
                      vector_w2v = vector_w2v / n_w2v
                  if n_ft > 0:
                      vector_ft = vector_ft / n_ft
                  w2v_index.add_item(counter, vector_w2v)
                  ft_index.add_item(counter, vector_ft)

                  counter += 1

                  if counter > 100000:
                      break

          w2v_index.build(10)
          ft_index.build(10)

```

```

Ввод [ ]: def get_response(question, index, model, index_map):
          question = preprocess_txt(question)
          vector = np.zeros(300)
          norm = 0
          for word in question:
              if word in model.wv:
                  vector += model.wv[word]
                  norm += 1
          if norm > 0:
              vector = vector / norm
          answers = index.get_nns_by_vector(vector, 3, )
          return [index_map[i] for i in answers]

```

```

Ввод [ ]: TEXT = "какой город самы красивый"

```

```

Ввод [ ]: get_response(TEXT, w2v_index, modelW2V, index_map)

```

```

Ввод [ ]: get_response(TEXT, ft_index, modelFT, index_map)

```

Ввод [ ]:

Ввод [ ]: